

PROGRAMMING ASSIGNMENT ANALYSIS

Jules Wygas, 40312026, jules.wygas@gmail.com

Clement Lepage, 40308494, cl_lepag@live.concordia.ca

Note: Requirements not implemented are omitted (Req. 4 and Req. 6)

Requirement 0 - Loading data

Description

We need to load all 14 million lines from the Bixi 2025 file into manageable and organized data structures as well as counting the total number of (valid) trips and different stations.

We decided to create a BixiTrip object for each line and store a reference to each object in each simple table where they were organized by a useful attribute to accomplish the later requirements more efficiently.

We only use the useful data in the files (discard latitude and longitude, shorten unix time stamps to seconds and use a custom dictionary to map station and arrondissement names to integers). We also use the smallest possible data type for each attribute and pre-compute the day of the year and hour of the day each trip occurred.

Objects are then created from a parsed line and pushed to all simple tables, where each one uses a different number of buckets and mapping, for example dayTable uses 355 buckets for the day the trip started on, this is useful for identifying the number of trips in each month later.

<i>Input</i>	Path of input file (N is number of lines)
<i>Output</i>	Creation of Bixitrip object and following tables:
<i>Implemented (Yes/No)</i>	Implemented by both

Complexity Analysis

Complexity analysis of each step of the algorithm.

Steps	Complexity
Instantiate all tables and fill with empty linked lists	$O(1)$
Creates dictionaries for stations and arrondissement from txt files	$O(1)$
File is loaded from the given path	$O(1)$
Each line is read, split, processed, an object is created and it is pushed to every table	$O(n)$
Total	$O(n)$

Analysis

Time to load all data for batch sizes:

Number or rows (n)	Time taken (seconds)
150	0
670,000	2
14,000,000	40

The scaling factor between our medium and large files is 20.9, matching the time scaling of 20. The parsing and processing step must be linear at best by nature of needing to traverse a file with N lines.

Requirement 1 - List all trips starting or ending in a given station

Description

We use our dictionary to convert the station name to an index, and then search in the startTable, endtable or both depending on the second user input. We return the linked list in that table at the index from the StationDict. If 'both' is selected, we fetch from both tables and use the custom linked list append function which takes the tail of the first linked list and points it at the head of the next, chaining them.

<i>Input</i>	Station name, start/end/both
<i>Output</i>	Expected response of the algorithm.
<i>Implemented (Yes/No)</i>	Clement

Complexity Analysis

Complexity analysis of each step of the algorithm.

Steps	Complexity
Use stationDict.getID() to get id of station	$O(1)$
Get linked list(s) from startStatTable and endStatTable as needed	$O(1)$
Append if needed	$O(n)$
TOTAL	$O(n)$

Analysis

Most of this function is $O(1)$ using our dictionary and simple table but unfortunately the linked list append() is $O(n)$ where n is the size of the first linked list. This is because we need to traverse the list to find its tail and make `tail.next = List2.head`, making the worse case for this requirement $O(n)$

Requirement 2 - List trips by month

Description

Parse input and verify year and month are valid. We use our lengthOfMonth array and a loop to find where in our dateTable our month starts and how long it lasts. We create a list MonthTrips and loop to append all our trips to it.

<i>Input</i>	Month in the “YYYY-MM” format
<i>Output</i>	Expected response of the algorithm.
<i>Implemented (Yes/No)</i>	Clement

Complexity Analysis

Complexity analysis of each step of the algorithm.

Steps	Complexity
Verify inputs	O(1)
Find start and length of month	O(1)
Repeatedly append from dateTable to monthTrips	O(N)
TOTAL	O(N)

Analysis

The appends take longer to do if there are more elements in each day, making busier months take longer to sum up; O(N) complexity.

Requirement 3 - List trips by minimum duration

Description

We first decided to establish a maximum accepted duration, this allows the majority of the data to be used while also removing what are likely to be recorded trips from stolen bixi's. We set this limit to a very lax 10 080 minutes ($24 * 60 * 7$). We calculate the duration making sure to convert it into minutes during the file parsing and insert the valid trips in the appropriate bucket at the index corresponding to the duration of that trip. Trips are then pushed back into a linked list at that index. `getTripsByDuration()` returns a newly made List which contains all trips in all lists from all buckets after `durationTable[min_duration]`.

<i>Input</i>	Min_duration (minute before all trips to be returned)
<i>Output</i>	List result (containing all buckets after min_duration)
<i>Implemented (Yes/No)</i>	Yes, by Jules Wygas

Complexity Analysis

Complexity analysis of each step of the algorithm.

Steps	Complexity
Step 1 Compute the duration threshold	$O(1)$
Step 2 Initialize result list	$O(1)$
Step 3 Traverse the nonEmptyIndices array to locate buckets with duration > X	$O(n)$
Step 4 Traverse each bucket and collect the trips	$O(D)$ (D = number of distinct durations)
Step 5 Insert each trip into the result list	$O(T)$ (T = number of trips being inserted)
TOTAL	$O(D + T)$

Analysis

The algorithm is fairly performant, having 2 very re-usable portions of code being $O(1)$. It however can be a lot more optimal, seeing as it currently depends on 2 separate variables (D and T).

Requirement 5 - Top k arrondissements

Description

When parsing the csv, arrondissement objects are stored in arrondissementTable using their corresponding dictionary ID. They hold names and trip counts. When parsing and adding trips to the table, arrondissement counts are incremented. Arrondissements have been sorted in top to descending order when creating the dictionary for each arrondissement. A for loop then returns the first k arrondissements in the table. The txt file is already sorted by each arrondissement, so no processing is needed.

<i>Input</i>	K (how many top arrondissements we want to return)
<i>Output</i>	Returns a list of Arrondissements objects
<i>Implemented (Yes/No)</i>	Yes, by Clement

Complexity Analysis

Complexity analysis of each step of the algorithm.

Steps	Complexity
Use order of txt file, return first k	O(1)
TOTAL	O(1)

Analysis

Because the arrondissements are sorted before hand, the implementation is quite efficient.

Requirement <<7>>

After checking that the user inputted a valid month, we return the start of the chosen month and the number of days in said month. Initialize the hourCount[24] array. Using a for loop, we go over every day of that month, return the link list present in dayTable on that day and count the trips at every hour of every day in the list. We then do a simple comparison to a best hour int and go through every hour in hourCount[] to find the one with the most trips. The function then returns a new RushHour object with the rushHour time and tripCount during that hour.

Description

Brief description of how the requirement was implemented.

<i>Input</i>	Given month (1 - 12)
<i>Output</i>	"Rush Hour: {rush hour}" + "Trips during rush hour: {numTrips}"
<i>Implemented (Yes/No)</i>	Yes, by Jules Wygas

Complexity Analysis

Complexity analysis of each step of the algorithm.

Steps	Complexity
Ensure month is valid	$O(1)$
Compute startDay, and daysInMonth	$O(1)$
Checks tripCounts for each hour of each day	$O(D)$ (D = Trips in a month)
Checks all 24 hours to find the hour with most trips	$O(1)$
TOTAL	$O(D)$

Analysis

The algorithm is well optimized in most steps, with its most costly one being $O(n)$. Not the best performance factor but good enough considering 3 of the 4 main steps are $O(1)$.

Requirement 8 - Compare 2 months

Description

Using a similar technique as the get month, we get all trips from the month interested, then run a rushHour() function on them. An array of stations is made and the number of trips in each station is added. We then use a cut off bubble sort to search k most used stations. This is output and repeated for month 2.

<i>Input</i>	2 months and k first stations
<i>Output</i>	Console description
<i>Implemented (Yes/No)</i>	Clement

Complexity Analysis

Complexity analysis of each step of the algorithm.

Steps	Complexity
Find month	$O(1)$
Repeated appends	$O(N)$
Sort	$O(N)$
TOTAL	$O(N)$

Analysis

The repeated appends take $O(N)$ as with getMonth. The truncated bubble sort is only $k \cdot n$ elements, simplifying to $O(N)$. All other steps are $O(1)$.